CHEATSHEET AA

Capitole

- Complexitatea algoritmilor (lab 2-4)
- Corectitudinea algoritmilor
- Clase de probleme
- Algoritmi de aproximare
- Calculabilitate

COMPLEXITATEA ALGORITMILOR

Definiţie: **Complexitatea unui algoritm** se referă la cantitatea de resurse de calcul (timp şi spaţiu de memorie) necesare execuţiei algoritmului.

Ce complexități măsurăm?

- cazul cel mai favorabil
- cazul cel mai defavorabil
- cazul mediu
- complexitate amortizată

Cum măsurăm complexitatea?

- exact: **toate operațiile** (complexitate exactă), **operațiile critice** (o aproximare a complexității exacte)
- aproximativ - **clase de complexitate**

Măsurarea exactă a complexității

Metrica (exactă) stabilește cantitatea de resurse folosite de operațiile elementare al_e unui algoritm

Metrica unitară și omogenă - fiecare operație elementară a unui algoritm cere o unitate convențională de timp (spațiu)

Metrica neomogenă - fiecare operație a unui algoritm folosește o anumită cantitate de resurse măsurată folosind unități convenționale de timp (spațiu).

Operații critice = operațiile executate cele mai frecvent sau cele mai complicate din punct de vedere al complexității

Exemple:

op1 (O(n)) for i = 1 .. n op2 (O(1))	op1 (O(n)) for i = 1 .. n op2 (O(n))	op1 ($O(n^2)$) for i = 1 .. n op2 (O(1))
operaţii critice: op1, op2	operaţii critice: op2	operaţii critice: op1

Complexitate - sortarea prin inserție

ins(v)	cost	repetiţii
for i = 2 .. n	c_1	n
aux = v[i]	c_2	$n-1$
j = i – 1	c_3	$n-1$
while j >= 1 and v[j] > aux	c_4	$\sum_{i=2}^{n} t_i$
v[j + 1] = v[j]	c_5	$\sum_{i=2}^{n} (t_i - 1)$
j = j – 1	c_6	$\sum_{i=2}^{n} (t_i - 1)$
v[j + 1] = aux	c_7	$n-1$

Toate operațiile

$$T(n) = c_1 \cdot n + c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot \sum_{i=2}^{n} t_i + \\ + c_5 \cdot \sum_{i=2}^{n} (t_i - 1) + c_6 \cdot \sum_{i=2}^{n} (t_i - 1) + c_7 \cdot (n-1)$$

Cazul cel mai favorabil ($t_i = 1$)

$$T(n) = c_1 \cdot n + c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot (n-1) + c_7 \cdot (n-1) = \\ = (c_1 + c_2 + c_3 + c_4 + c_7) \cdot n - (c_2 + c_3 + c_4 + c_7) = \Theta(n)$$

Formatted: Font: 10 pt

Commented [1]: materie de kkt

Deleted:

Cazul cel mai defavorabil ($t_i = i$)

$$T(n) = c_1 \cdot n + c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot \sum_{i=2}^{n} i +$$
$$+ c_5 \cdot \sum_{i=2}^{n} (i-1) + c_6 \cdot \sum_{i=2}^{n} (i-1) + c_7 \cdot (n-1) = \ldots =$$
$$= \left(\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2}\right) \cdot n^2 + (c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7) \cdot n -$$
$$- (c_2 + c_3 + c_4 + c_7) = \Theta(n^2)$$

$$\left(t_{i-1} = \frac{0 + (i-1)}{2} \Rightarrow t_i = \frac{i+1}{2} \right):$$

Cazul mediu

$$T(n) = c_1 \cdot n + c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot \sum_{i=2}^{n} \frac{i+1}{2} +$$
$$+ c_5 \cdot \sum_{i=2}^{n} \frac{i-1}{2} + c_6 \cdot \sum_{i=2}^{n} \frac{i-1}{2} + c_7 \cdot (n-1) = \ldots =$$
$$= \left(\frac{c_4}{4} + \frac{c_5}{4} + \frac{c_6}{4}\right) \cdot n^2 + (c_1 + c_2 + c_3 + \frac{3 \cdot c_4}{4} - \frac{c_5}{4} - \frac{c_6}{4} + c_7) \cdot n -$$
$$- (c_2 + c_3 + c_4 + c_7) = \Theta(n^2)$$

Operații critice

Cazul cel mai favorabil - op. crit. = 2, 3, 4, 7

$$T(n) = c_2 \cdot (n-1) + c_3 \cdot (n-1) + c_4 \cdot \sum_{i=2}^{n} 1 + c_7 \cdot (n-1) =$$
$$= \ldots = (c_2 + c_3 + c_4 + c_7) \cdot n - (c_2 + c_3 + c_4 + c_7) = \Theta(n)$$

Cazul cel mai defavorabil, op critice = 5, 6

$$T(n) = c_5 \cdot \sum_{i=2}^{n} (i-1) + c_6 \cdot \sum_{i=2}^{n} (i-1) = \ldots =$$
$$= \left(\frac{c_5}{2} + \frac{c_6}{2}\right) \cdot n^2 - \left(\frac{c_5}{2} + \frac{c_6}{2}\right) \cdot n = \Theta(n^2)$$

Cazul mediu, op critice = 5, 6

$$T(n) = c_5 \cdot \sum_{i=2}^{n} \frac{i-1}{2} + c_6 \cdot \sum_{i=2}^{n} \frac{i-1}{2} = \ldots =$$
$$= \left(\frac{c_5}{4} + \frac{c_6}{4}\right) \cdot n^2 - \left(\frac{c_5}{4} + \frac{c_6}{4}\right) \cdot n = \Theta(n^2)$$

Clase de complexitate

$O(g(n)) = \{ f : \mathbb{N}^* \to \mathbb{R}_+^* \mid \exists c \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$ a.î. $f(n) \le c \cdot g(n), \forall n \ge n_0 \}$

$\Omega(g(n)) = \{ f : \mathbb{N}^* \to \mathbb{R}_+^* \mid \exists c \in \mathbb{R}_+^* \exists n_0 \in \mathbb{N}^*$ a.î. $f(n) \ge c \cdot g(n), \forall n \ge n_0 \}$

$\Theta(g(n)) = \{ f : \mathbb{N}^* \to \mathbb{R}_+^* \mid \exists c_1, c_2 \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$ a.î. $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n), \forall n \ge n_0 \}$

$o(g(n)) = \{ f : \mathbb{N}^* \to \mathbb{R}_+^* \mid \forall c \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$ a.î. $f(n) < c \cdot g(n), \forall n \ge n_0 \}$

$\omega(g(n)) = \{ f : \mathbb{N}^* \to \mathbb{R}_+^* \mid \forall c \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$ a.î. $f(n) > c \cdot g(n), \forall n \ge n_0 \}$

Notație pentru clasele de complexitate

Tradițional se notează $f(n) = X(g(n))$ cu $X \in \{O, \Omega, \Theta, o, \omega\}$

Proprietăți clase de complexitate

- limită

Proprietăți

$f(n) = o(g(n)) \Leftrightarrow \lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$

$f(n) = \omega(g(n)) \Leftrightarrow \lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$

Propoziția 1:

a) $o(g(n)) \subseteq O(g(n))$

b) $\omega(g(n)) \subseteq \Omega(g(n))$

- tranzitivitatea (se aplică pentru pt toate clasele de complexitate)

$f(n) = O(g(n))$, $g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$

- reflexivitatea (toate în afară de omega mic și o mic)

$f(n) = O(f(n))$

- simetria(θ)

Formatted: Font: 10 pt

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: 10 pt, Not Bold

Deleted: $O(g(n)) = \{ f : \mathbb{N}^* \to \mathbb{R}_+^* \mid \exists c \in \mathbb{R}_+^*, \exists n_0 \in$ a.î. $f(n) \le c \cdot g(n), \forall n \ge$

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted ... [1]

Formatted ... [2]

Formatted ... [3]

Formatted ... [4]

Formatted ... [5]

Formatted ... [6]

Formatted ... [7]

Formatted ... [8]

$f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$

- simetria transpusă $(O, \Omega), (o, \omega)$
- proiecția $(\Theta, (O, \Omega))$

$f(n) = \Theta(g(n)) \Leftrightarrow \begin{cases} f(n) = O(g(n)) \\ \text{și} \\ f(n) = \Omega(g(n)) \end{cases}$

- pentru

$f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f($
$f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f($

(O, Ω, Θ) se aplică

a) $k \cdot \Theta(f(n)) = \Theta(k \cdot f(n)) = \Theta(f(n))$
b) $\Theta(f(n)) + \Theta(g(n)) = \Theta(f(n) + g(n))$
c) $\Theta(f(n)) * \Theta(g(n)) = \Theta(f(n) * g(n))$
d) $\Theta(c_k \cdot n^k + c_{k-1} \cdot n^{k-1} + \dots + c_0) = \Theta(n^k)$, $c_k > 0$.

Metode de rezolvare a recurentelor de complexitate

- Metoda iterativă
- Metoda arborelui de recurență
- Metoda substituției
- Metoda Master

Notație: $\log_2 n \overset{not}{=} \lg n$

$n = 2^{k+1}$
$k+1 = \lg n$
$k = \lg n - 1$

Metoda iterativă

Ex 1:

$$T(n) = \begin{cases} k_1, & n = 1 \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + k_2 n, & n > 1 \end{cases}$$

$\Rightarrow T(n) = 2T(n/2) + \Theta(n)$,
$T(1) = \Theta(1)$

$T(n) = 2T(n/2) + \Theta(n) \quad | \cdot 2^0$
$T(n/2) = 2T(n/2^2) + \Theta(n/2) \quad | \cdot 2^1$
$T(n/2^2) = 2T(n/2^3) + \Theta(n/2^2) \quad | \cdot 2^2$
$+$
$\dots$
$T(n/2^k) = 2T(n/2^{k+1}) + \Theta(n/2^k) \quad | \cdot 2^k$

$T(n) = 2^{k+1} \cdot T(n/2^{k+1}) + \sum_{i=0}^{k} 2^i \cdot \Theta(n/2^i) =$

$n = 2^{k+1} \Rightarrow k+1 = \lg n \Rightarrow k = \lg n - 1$

$= n \cdot T(1) + (k+1) \cdot \Theta(n) =$
$= n \cdot \Theta(1) + \lg n \cdot \Theta(n) =$
$= \Theta(n \lg n + n) = \Theta(n \lg n)$

după ce am obținut clasa de complexitate o putem aduce la o formă mai simplă

-> Notații folosite

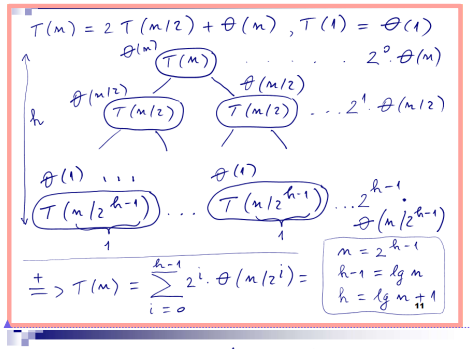
Metoda arborelui de recurență

Ex 1:

$$T(n) = \begin{cases} k_1, & n = 1 \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + k_2 n, & n > 1 \end{cases}$$

$\Rightarrow T(n) = 2T(n/2) + \Theta(n)$,
$T(1) = \Theta(1)$

Formatted ... [9]
Formatted ... [10]
Formatted ... [11]
Formatted ... [23]
Formatted ... [35]
Formatted ... [24]
Formatted ... [12]
Formatted ... [25]
Deleted: $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f($ $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f($
Formatted ... [13]
Formatted ... [26]
Formatted ... [14]
Formatted ... [27]
Formatted ... [28]
Formatted ... [29]
Formatted ... [15]
Formatted ... [16]
Formatted ... [36]
Formatted ... [30]
Formatted ... [31]
Formatted ... [17]
Formatted ... [18]
Formatted ... [32]
Formatted ... [19]
Formatted ... [20]
Formatted ... [37]
Formatted ... [33]
Formatted ... [34]
Deleted: \
Formatted ... [38]
Formatted ... [39]
Formatted ... [21]
Formatted ... [22]



$= h \cdot \Theta(n) = (\lg n + 1) \cdot \Theta(n) =$

$= \Theta(n \lg n + n) = \Theta(n \lg n)$

$T(n) = 2T(n/2) + k_2 n$, $T(1) = k_1$

Alegem $T(n) = \Theta(n \lg n + n)$

$\exists c_1, c_2 \in \mathbb{R}_+^*$, $\exists n_0 \in \mathbb{N}^*$ a.î.

$c_1(n \lg n + n) \le T(n) \le c_2(n \lg n + n)$, $\forall n \ge n_0$

Demonstrăm prin inducție inegalitățile de mai sus.

$T(n) = 2T(n/2) + k_2 n$, $T(1) = k_1$

$c_1(n \lg n + n) \le T(n) \le c_2(n \lg n + n)$

Caz de bază

$n = 1$: $c_1 \le k_1 \le c_2$ (1) $\Rightarrow n_0 = 1$

Pas de inducție: $n/2 \to n$

Ipoteza de inducție:

$c_1\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right) \le T\left(\frac{n}{2}\right) \le c_2\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right)$

Arătăm că:

$c_1(n \lg n + n) \le T(n) \le c_2(n \lg n + n)$

$T(n) = 2T(n/2) + k_2 n$, $T(1) = k_1$

$c_1(n \lg n + n) \le T(n) \le c_2(n \lg n + n)$

$c_1\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right) \le T\left(\frac{n}{2}\right) \le c_2\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right)$

$2c_1\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right) + k_2 n \le 2T\left(\frac{n}{2}\right) + k_2 n \le$

$\le 2c_2\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right) + k_2 n$

$c_1 n \lg \frac{n}{2} + c_1 n + k_2 n \le T(n) \le$

$\le c_2 n \lg \frac{n}{2} + c_2 n + k_2 n$

$c_1 n \lg n - c_1 n + c_1 n + k_2 n \le T(n) \le$

$\le c_2 n \lg n - c_2 n + c_2 n + k_2 n$

$c_1(n \lg n + n) + (k_2 - c_1) \cdot n \le T(n) \le$

$c_1(n \lg n + n) \le$ ($k_2 - c_1 \ge 0$)

$\le c_2(n \lg n + n) + (k_2 - c_2) \cdot n$ ($k_2 - c_2 \le 0$)

$\le c_2(n \lg n + n)$

$\Rightarrow \begin{cases} k_2 \ge c_1 \\ k_2 \le c_2 \end{cases} \Rightarrow c_1 \le k_2 \le c_2$ (2)

Caz de bază: $c_1 \le k_1 \le c_2$, $n_0 = 1$

Pas de inducție: $c_1 \le k_2 \le c_2$

$\Rightarrow \exists c_1 = \min(k_1, k_2), c_2 = \max(k_1, k_2) \in \mathbb{R}_+^*$,

$\exists n_0 = 1 \in \mathbb{N}^*$ a.î.

$c_1(n \lg n + n) \le T(n) \le c_2(n \lg n + n)$, $\forall n \ge n_0$

$\Rightarrow T(n) = \Theta(n \lg n + n)$

Metoda substituției

- se alege clasa de complexitate (soluția)
- se folosește inducția matematică pentru a afla constantele (pentru clasa de complexitate) și pentru a arăta că soluția verifică formula de recurență

Metoda Master

Formatted: Font: 10 pt

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: 10 pt, Not Bold

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: 10 pt, Not Bold

T este de forma

$$T(n) = aT(n/b) + f(n)$$

Se disting 3 cazuri:

1. Dacă $\exists \varepsilon > 0$ a.î. $f(n) = O(n^{\log_b a - \varepsilon})$ atunci $T(n) = \Theta(n^{\log_b a})$

2. Dacă $f(n) = \Theta(n^{\log_b a})$ atunci $T(n) = \Theta(n^{\log_b a} \cdot \lg n)$

3. Dacă $\exists \varepsilon > 0$ a.î. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ și $\exists c \in (0,1)$, $n_0 \in \mathbb{N}^*$ a.î. $af(n/b) \leq cf(n)$, $\forall n \geq n_0$ atunci $T(n) = \Theta(f(n))$

20

! Metoda Master nu poate fi aplicată mereu

Ordinea în care se verifică cazurile este: 2-1-3

Exemplu de rezolvare folosind cazul 1 (se va verifica și cazul 2 înainte).

Exemplu de rezolvare folosind cazul 2

Ex 2: $T(n) = T(2n/3) + \Theta(1)$

$T(n) = aT(n/b) + f(n)$

caz 2: Dacă $f(n) = \Theta(n^{\log_b a})$ atunci $T(n) = \Theta(n^{\log_b a} \cdot \lg n)$

$a = 1$, $b = 3/2$ $\Rightarrow n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$

$f(n) = \Theta(1) = \Theta(n^{\log_b a})$

caz 2 $\Rightarrow T(n) = \Theta(n^{\log_b a} \cdot \lg n) = \Theta(\lg n)$

Exemplu de rezolvare folosind cazul 3 (se vor verifica și cazurile 2-1 înainte)

Ex 3: $T(n) = 3T(n/4) + n \lg n$

$T(n) = aT(n/b) + f(n)$

caz 3: Dacă $\exists \varepsilon > 0$ a.î. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ și $\exists c \in (0,1)$, $n_0 \in \mathbb{N}^*$ a.î. $af(n/b) \leq cf(n)$, $\forall n \geq n_0$ atunci $T(n) = \Theta(f(n))$

$a = 3$, $b = 4$ $\Rightarrow n^{\log_b a} = n^{\log_4 3} = n^{\log_2 3 / \log_2 4} = n^{(\lg 3)/2}$

$f(n) = n \lg n$

24

a) $\exists \varepsilon > 0$ a.î. $f(n) = \Omega(n^{\log_b a + \varepsilon})$

Bem

$\exists c' \in \mathbb{R}_+^*$, $\exists n_0' \in \mathbb{N}^*$ a.î.

$c' n^{\log_b a + \varepsilon} \leq f(n)$, $\forall n \geq n_0'$

$c' n^{(\lg 3)/2 + \varepsilon} \leq n \lg n$, $\forall n \geq n_0'$

$\lg n \geq c' n^{(\lg 3)/2 - 1 + \varepsilon}$, $\forall n \geq n_0'$

$(1 > (\lg 3)/2 \Rightarrow 2 > \lg 3 \Rightarrow 2^2 > 3$ adev)

$\Rightarrow \varepsilon = 1 - (\lg 3)/2 > 0$

$\Rightarrow \lg n \geq c'$, $\forall n \geq n_0' \Rightarrow \begin{cases} n_0' = 2 \in \mathbb{N}^* \\ c' = 1 \in \mathbb{R}_+^* \end{cases}$

Formatted: Font: 10 pt

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

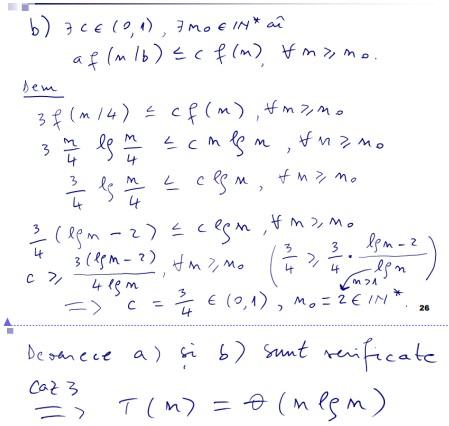
Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Font: Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Formatted: Pattern: Clear (Custom Colour (RGB(255,200,156)))

Deleted: ¶ ... [40]

Formatted ... [41]



Deoarece a) și b) sunt verificate

caz 3

$\Rightarrow T(m) = \Theta(m \lg m)$

Dacă nu se poate aplica teorema Master, aplicăm o altă metodă pentru a rezolva recurența.

Analiză amortizată

- generează performanța medie (costul amortizat) pentru fiecare operație din secvență, în cazul cel mai defavorabil
- costul amortizat total reprezintă suma costurilor amortizate ale operațiilor din secvență

cost amortizat total al secvenței ≥ cost real total al secvenței în cazul cel mai defavorabil

Metode pentru analiza amortizată

- metoda analizei de ansamblu
- metoda creditelor
- metoda potențialului

Metoda analizei de ansamblu

- arătăm că pentru orice m, o secvență de m operații are costul T(m) în cazul cel mai defavorabil
- costul amortizat al unei operații este T(m) / m, unde T(m) este costul amortizat total
- toate operațiile au același cost amortizat

Metoda creditelor

- fiecare operație are un cost amortizat și un cost real pe care îl asociem la începutul analizei
- costa(op) = costul amortizat al operației op
- cost(op) = costul real al operației op
- dacă costa(op) > cost(op) atunci diferența numită credit este asociată unor obiecte din structura de date
- dacă costa(op) < cost(op) atunci diferența este plătită folosind credite din structura de date

- $cost_a(op) = cost(op) + credit$ (pus în structura de date sau lucrat de acolo)
- cost amortizat total = cost real total + $\sum$ credite structură

Metode potențialului

- potențial asociat întregii structuri de date
- potențialul va fi folosit pentru a plăti diferite operații

- n operații: $op_1, op_2, \dots, op_n$
- $cost_a(op_i)$ = costul amortizat al operației op_i
- $cost(op_i)$ = costul real al operației op_i

Formatted ... [42]
Formatted ... [43]
Formatted ... [48]
Formatted ... [49]
Formatted ... [64]
Formatted ... [65]
Formatted ... [50]
Formatted ... [66]
Deleted: ¶
Formatted ... [51]
Formatted ... [52]
Formatted ... [53]
Formatted ... [54]
Formatted ... [44]
Formatted ... [67]
Formatted ... [55]
Formatted ... [68]
Formatted ... [56]
Formatted ... [69]
Formatted ... [57]
Formatted ... [70]
Formatted ... [71]
Formatted ... [58]
Formatted ... [45]
Formatted ... [46]
Formatted ... [59]
Formatted ... [60]
Formatted ... [47]
Formatted ... [61]
Formatted ... [62]
Formatted ... [63]

ϕ = funcția de potențial (se alege în funcție de problemă)

D_0 = structura inițială

D_i = structura rezultată după aplicarea operației op_i

$\phi_i = \phi(D_i)$ = potențialul asociat structurii D_i

$$\begin{cases} D_i = op_i(op_{i-1}(\dots op_1(D_0)\dots)) = op_i(D_{i-1}), \\ \phi_i = \phi(D_i), \end{cases} \quad i = \overline{1,m}$$

$cost_a(op_i) = cost(op_i) + \phi_i - \phi_{i-1}, \; i = \overline{1,m}$

$$\sum_{i=1}^{m} cost_a(op_i) = \sum_{i=1}^{m} (cost(op_i) + \phi_i - \phi_{i-1}) =$$

$$= \sum_{i=1}^{m} cost(op_i) + \phi_m - \phi_0$$

$$\sum_{i=1}^{m} cost_a(op_i) \geq \sum_{i=1}^{m} cost(op_i) \Rightarrow \boxed{\phi_m \geq \phi_0}$$

cost amortizat total — cost real total

Variante

- $\phi_m \geq \phi_0$ - greu de aplicat
- $\phi_i \geq \phi_0, \forall i \quad (\phi_0 = 0 \Rightarrow \phi_i \geq 0, \forall i)$
- $\phi_i \geq \phi_{i-1}, \forall i$

Corectitudinea algoritmilor

Considerăm un algoritm Alg cu date de tip I și rezultate de tip O.

- Datele $d \in I$ sunt considerate corecte dacă satisfac un predicat $P_I : I \to \{0,1\}$ numit asertiune de intrare.

- Rezultatele algoritmului pentru datele $d \in I$, rezultate notate cu $r = Alg(d)$, $r \in O$, sunt considerate corecte dacă satisfac un predicat $P_O : I \times O \to \{0,1\}$ numit asertiune de ieșire

Cele două asertiuni (asertiune de intrare și de ieșire) caracterizează problema rezolvată.

Ex - factorialul unui număr

$I = \mathbb{N}$

$O = \mathbb{N}$

```
fact(n)
    if n = 1
        return 1
    else
        return n * fact(n - 1)
```

$$P_I(n) =_{def} n \geq 1$$

$$P_O(n, r) =_{def} r = \prod_{k=1}^{n} k$$

Parțial corectitudine (în raport cu asertiunile P_I și P_O)

Dacă

$P_I(d) = 1$ și se_termină(Alg, d)

rezultă că

$P_O(d, Alg(d)) = 1.$

- demonstrăm ca asertiunea de ieșire este adevărată pentru orice date care satisfac asertiunea de intrare

ϕ = funcția de potențial (se alege în funcție de problemă)

D_0 = structura inițială

D_i = structura rezultată după aplicarea operației op_i

$\phi_i = \phi(D_i)$ = potențialul asociat structurii D_i

$$\begin{cases} D_i = op_i(op_{i-1}(\dots op_1(D_0)\dots)) = op_i(D_{i-1}), \\ \phi_i = \phi(D_i), \end{cases} \quad i = \overline{1,m}$$

Deleted:

Formatted: Font: 10 pt

Formatted ... [80]
Formatted ... [81]
Formatted ... [75]
Formatted ... [76]
Formatted ... [82]
Formatted ... [83]
Formatted ... [72]
Formatted ... [73]
Formatted ... [84]
Formatted ... [85]
Formatted ... [77]
Formatted ... [86]
Formatted ... [87]
Formatted ... [88]
Formatted ... [78]
Formatted ... [79]
Formatted ... [74]
Formatted ... [89]
Formatted ... [90]

- terminarea unui algoritm o vom considera fie evidentă, fie o vom explica intuitiv
- pentru orice intrare corectă a algoritmului pentru care algoritmul se termină, ieșirea algoritmului este corectă

Total corectitudine (în raport cu aserțiunile P_I și P_O)

Dacă

$d \in I$ cu proprietatea că

$P_I(d) = 1$

rezultă că

$\mathcal{A}$ - termină(Alg, d) și $P_O(d, Alg(d)) = 1$.

!! Pentru a analiza corectitudinea parțială și corectitudinea totală a unui algoritm trebuie să se ia în considerare atât algoritmul respectiv, cât și problema care trebuie rezolvată.

Analiza corectitudinii unui algoritm nu are sens fără existența unei aserțiuni de intrare și a unei aserțiuni de ieșire care descrie problema.

Metode pentru verificarea corectitudinii parțiale

1. Deducție
   - directă
   - prin reducere la absurd
2. Inducție bine formată
   - inducție completă
   - inducție matematică
   - inducție structurală

Deducție

Deducția directă

ipoteză ... => ... concluzie

Deducția prin reducere la absurd

not (concluzie) ... => ... not (ipoteză)

Inducția bine formată

Relație bine formată

Def: Fie $<$ o relație peste mulțimea X astfel încât orice secvență

$\ldots < x_n < \ldots < x_2 < x_1$

este finită. Spunem că relația este bine formată.

Exemple

- < pe N este bine formată, dar < pe Z nu este bine formată (există o infinitate de numere întregi mai mici decât un număr arbitrar x din Z)
- <= nu este bine formată (x1 <= x1 <= .... <= x1 - secvență infinită)

Funcție de măsurare

- f:X-> N= funcția de măsurare a elementelor din X
- relația bine formată <f peste X este:

$\forall a, b \in X,\ a <_f b \Leftrightarrow f(a) < f(b)$.

- relația bine formată a <f b în X devine f(a) < f(b) în N

Inducția bine formată se bazează pe faptul că relația < peste N este bine formată.

Formatted ... [91]
Formatted ... [92]
Formatted ... [104]
Formatted ... [121]
Formatted ... [122]
Formatted ... [105]
Formatted ... [106]
Formatted ... [93]
Formatted ... [107]
Formatted ... [108]
Formatted ... [109]
Formatted ... [94]
Formatted ... [110]
Formatted ... [123]
Formatted ... [111]
Formatted ... [95]
Formatted ... [96]
Formatted ... [112]
Formatted ... [124]
Formatted ... [97]
Formatted ... [113]
Formatted ... [98]
Formatted ... [99]
Formatted ... [114]
Formatted ... [125]
Formatted ... [115]
Formatted ... [116]
Formatted ... [126]
Formatted ... [127]
Formatted ... [100]
Formatted ... [101]
Formatted ... [117]
Formatted ... [118]
Formatted ... [128]
Formatted ... [129]
Formatted ... [130]
Formatted ... [131]
Formatted ... [102]
Formatted ... [119]
Formatted ... [103]
Formatted ... [120]
Formatted ... [132]

Fie $B = \{x \in X \mid \nexists y \in X$ astfel încât $y <_f x\}$

mulțimea de bază a inducției (conține cazurile de bază)

$X = IN$, $<_f = <$, $B = \{0\}$

Caz de bază: $P(0)$

Pas de inducție: $\forall n \in IN^*$ $\dfrac{\forall i \in IN \text{ de } i < n \text{ at } P(i)}{P(n)}$

$\forall n \in IN$ avem $P(n)$

Caz de bază: $P(m_0)$

Pas de inducție: $\forall m_i \geq m_0$ $\dfrac{P(m_i)}{P(g(m_i))}$

$\forall n \in IN$, $n = g^k(m_0)$ avem $P(n)$

Obs:

Se observă că $B = \{m_0\}$.

Schema inducției bine formate

Caz de bază

$\forall x \in B$ avem $P(x)$

Pas de inducție: $\dfrac{\text{(ipoteza de inducție) } \forall y \in X, \text{ de } y <_f x \text{ at. } P(y)}{P(x)}$

$\forall x \in X \setminus B$

Caz de bază + pas de inducție $\Rightarrow$ $\forall x \in X$ avem $P(x)$

Inducția completă

Inducția matematică

$X = IN$, $x <_f y \Leftrightarrow x + 1 = y$, $B = \{0\}$

Caz de bază: $P(0)$

Pas de inducție: $\forall n \in IN$ $\dfrac{P(n)}{P(n+1)}$

$\forall n \in IN$ avem $P(n)$

Generalizare inducție matematică

$X = IN$, $x <_f y \Leftrightarrow y = g(x)$, unde $g : IN \to IN$, $g(x) > x$, $\forall x \in IN$.

Notăm $g^k(m_0) = \underbrace{g(\ldots g(}_{\text{de } k \text{ ori}} g(m_0) \ldots)$, $g^0(m_0) = m_0$.

Considerăm m_0 limita inferioară a secvenței $m_0 < m_1 < \ldots < m_i < m_{i+1} < \ldots$ unde $m_{i+1} = g(m_i)$.

Inducția structurală

X = t, t = tip de date

$x \in t$, $f(x)$ = adâncimea maximă a aparițiilor constructorilor lui t în cadrul valorii simbolice x.

$\Sigma_0 =_{def} \{\sigma : \to t\}$ → mulțimea constructorilor nulari ai tipului t

$\Sigma_e =_{def} \{\sigma : Dom_\sigma \to t / t$ nu apare în $Dom_\sigma\}$ → mulțimea constructorilor externi ai tipului t

Formatted ... [133]
Formatted ... [134]
Formatted ... [135]
Formatted ... [144]
Formatted ... [145]
Formatted ... [156]
Formatted ... [157]
Formatted ... [158]
Formatted ... [159]
Formatted ... [136]
Formatted ... [146]
Formatted ... [137]
Formatted ... [138]
Formatted ... [147]
Formatted ... [148]
Formatted ... [160]
Formatted ... [161]
Formatted ... [162]
Formatted ... [163]
Formatted ... [149]
Formatted ... [139]
Formatted ... [140]
Formatted ... [150]
Formatted ... [151]
Formatted ... [164]
Formatted ... [165]
Formatted ... [152]
Formatted ... [153]
Formatted ... [166]
Formatted ... [167]
Formatted ... [141]
Formatted ... [142]
Formatted ... [154]
Formatted ... [155]
Formatted ... [143]

$\Sigma_t \stackrel{def}{=} \{\sigma : Dom_\sigma \to t \mid t \text{ apare în } Dom_\sigma\}$
→ mulțimea constructorilor interni ai tipului t

19

exemplu - pentru Tset = mulțimi de elemente de tip T

newset : → TSet // construiește mulțimea vidă

oneset : T → TSet // primește ca intrare un element de tip T și returnează mulțimea care conține doar acel element

insert : T × TSet → TSet // adaugă un element de tip T într-o mulțime de tip TSet

21

$\Sigma_0 = \{new\ set\}, \Sigma_2 = \{onset\}$

$\Sigma_3 = \{insert\}$

Schema inducției structurale

Caz de bază : $\forall \sigma \in \Sigma_0$ avem $P(\sigma)$

$\forall \sigma \in \Sigma_e, v \in Dom_\sigma$ avem $P(\sigma(v))$

Pas de inducție :

$$\frac{\forall (x_1, x_2, \dots, x_m) \in Dom_\sigma \text{ de } x_i \in t \text{ at } P(x_i)}{P(\sigma(x_1, x_2, \dots, x_m))}$$

$\forall \sigma \in \Sigma_t$

$\forall x \in t$ avem $P(x)$

- inducția structurală folosește dimensiunea valorilor simbolice ale tipului t

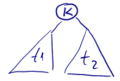
TTrece = arbori binari cu chei de tip T

Operatori

empty : → TTree // generează arborele vid

node : TTree × T × TTree → TTree
// construiește un arbore cu rădăcina de tip T și cu subarbori stâng și drept de tip TTree

De exemplu,
node (t1, K, t2) este arborele



25

Exemplu inducție structurală

m(t) // m de la mirror
if t = empty
  return empty
else if t = node (t1, K, t2)
  return node (m(t2), K, m(t1))

Să se arate că :
$\forall t \in TTree$ avem $m(m(t)) = t$

Utilizăm inducția structurală pentru a demonstra proprietatea cerută.

Caz de bază : t = empty

$m(m(t)) = m(m(empty)) \stackrel{m}{=} m(empty) \stackrel{m}{=} empty = t$

Pas de inducție : $t = node(t_1, K, t_2)$

Ipoteza de inducție :
$m(m(t_1)) = t_1, m(m(t_2)) = t_2$

Arătăm că : $m(m(t)) = t$

construg

$m(m(t)) = m(m(node(t_1, K, t_2))) \stackrel{m}{=}$
$= m(node(m(t_2), K, m(t_1))) \stackrel{m}{=}$
$= node(m(m(t_1)), K, m(m(t_2))) =$
$\stackrel{ip.ind.}{=} node(t_1, K, t_2) = t$

Formatted ... [168]
Formatted ... [169]
Formatted ... [170]
Formatted ... [185]
Formatted ... [186]
Formatted ... [194]
Formatted ... [195]
Formatted ... [171]
Formatted ... [172]
Formatted ... [173]
Formatted ... [174]
Formatted ... [187]
Formatted ... [196]
Formatted ... [197]
Formatted ... [188]
Formatted ... [189]
Formatted ... [190]
Formatted ... [191]
Formatted ... [192]
Formatted ... [193]
Formatted ... [175]
Formatted ... [176]
Formatted ... [177]
Formatted ... [178]
Formatted ... [179]
Formatted ... [180]
Formatted ... [181]
Formatted ... [182]
Formatted ... [183]
Formatted ... [184]

Caz de bază verificat
Pas de inducție verificat
=> ∀ t ∈ TTree avem m(m(t)) = t

TMSet = multiseturi cu elemente de tip T. Un multiset este o "mulțime' în care elementele pot avea apariții multiple

Operatori:

∅ : multisetul vid

x + S : inserția elementului x în multisetul S.
Dacă x apare în S de x_S ori atunci
x apare în x + S de $x_S + 1$ ori

S - x : eliminarea elementului x din multisetul S.
Dacă x apare în S de x_S ori atunci
x apare în S - x de $x_S - 1$ ori.

$S_1 \underline{\cup} S_2$: reuniunea multisetsurilor S_1 și S_2

x # S : numărul de apariții ale elementului x în S.

Constructori de bază și alți operatori

a) Constructori de bază

$\emptyset : \rightarrow TMset$

$+ : T \times TMset \rightarrow TMset$

b) Alți operatori

$- : TMset \times T \rightarrow TMset$

$\underline{\cup} : TMset \times TMset \rightarrow TMset$

$\# : T \times TMset \rightarrow \mathbb{N}$

Axiome

$\emptyset - e = \emptyset$ -primele 2 ecuații corespund axiomelor de bază

$(e' + S) - e = \text{if } (e = e') \text{ then } S \text{ else } e' + (S - e)$

$\emptyset \underline{\cup} S = S$

$(e + S) \underline{\cup} S' = e + (S \underline{\cup} S')$

$e \# \emptyset = 0$

$e \# (e' + S) = \text{if } (e = e') \text{ then } 1 + (e \# S) \text{ else } (e \# S)$

Tipul TArb + Mulțimile THeap și T arh

$TArb = A_\emptyset \mid arb(T, TArb, TArb)$

$THeap = \{A \in TArb \mid e_heap(A)\}$

$TArh = \{A \in TArb \mid A \neq A_\emptyset \Rightarrow e_heap(S(A)) \text{ și } e_heap(D(A))\}$

Constructori de bază

$A_\emptyset : \rightarrow TArb$ // arborele vid

$arb : T \times TArb \times TArb \rightarrow TArb$
// rădăcină, subarbore stâng, subarbore drept => nou arbore

Alți operatori

$S : TArb \setminus \{A_\emptyset\} \rightarrow TArb$ // subarborele stâng

$D : TArb \setminus \{A_\emptyset\} \rightarrow TArb$ // subarborele drept

$top : TArb \rightarrow T_\infty$ // rădăcina

$elm : TArb \rightarrow TMset$ // multisetul cu nodurile arborelui

$e_heap : TArb \rightarrow Bool$

$vern : TArh \rightarrow THeap$

Axiome

Formatted: Font: 10 pt

$S(arb(x, A_s, A_d)) = A_s$

$D(arb(x, A_s, A_d)) = A_d$

$top(A_\emptyset) = \infty$

$top(arb(x, A_s, A_d)) = x$

$elem(A_\emptyset) = \emptyset$

$elem(arb(x, A_s, A_d)) = x + (elem(A_s) \cup elem(A_d))$

$e_heap(A_\emptyset) = 1$

$e_heap(arb(x, A_s, A_d)) = (x \leq min(top(A_s), top(A_d))$ și $e_heap(A_s)$ și $e_heap(A_d)$

13

$cern(A_\emptyset) = A_\emptyset$

$cern(arb(x, A_s, A_d)) =$

if $x \leq min(top(A_s), top(A_d))$ then $arb(x, A_s, A_d)$

else

if $top(A_s) \leq top(A_d)$ then

$arb(top(A_s), cern(arb(x, s(A_s), D(A_s))), A_d)$

else

$arb(top(A_d), A_s, cern(arb(x, s(A_d), D(A_d))))$

$\forall A \in TArb$ avem $elem(cern(A)) = elem(A)$

CLASE DE PROBLEME

Def: Problema Q s.n. de decizie dacă, pentru o problemă (Q', I, O) dată, este de tipul: "Q' are soluție pentru $i \in I$?".

Def: Problema Q s.n. de optimizare dacă selectează soluțiile r ale unei probleme date (Q', I, O) ce optimizează (maximizează sau minimizează) o funcție de cost definită pe $I \times O$.

Determinism vs nedeterminism

Algoritm determinist -> fiecare operație de control sau de prelucrare a datelor are un rezultat unic determinat => **nu se execută operații în paralel**

Caracteristici

- seriabilitate
- la orice moment de timp se execută o singură operație

există o funcție Timp → Operații care descrie execuția algoritmului:

t → op1
→ op2

Algoritm nedeterminist -> are cel puțin o operație al cărei rezultat **nu este unic definit**, ci este o valoare dintr-o mulțime finită de posibilități (există operații care se execută în paralel)

Caracteristici

- paralelism nestricțional al algoritmului
- execuția unui algoritm nedeterminist conduce la o structură arborescentă de operații
- operațiile de pe o cale sunt executate serial, în timp ce operațiile de pe căi diferite sunt executate în paralel

- există o submulțime a produsului Timp x Operații care descrie execuția algoritmului: t → op1, op2

Operații algoritm nedeterminist

1. **choice(A)**

- în momentul execuției operației choice pentru fiecare valoare din A se creează o copie a algoritmului

2. **fail**

- termină cu insucces execuția copiei algoritmului în care se află. Celelalte copii nu sunt afectate

- întoarce valoarea 0

Formatted: Font: 10 pt

3. **success**

- termină cu succes execuția copiei algoritmului în care se află și a tuturor celorlalte copii
- întoarce valoarea 1

Un algoritm nedeterminist se termină cu succes dacă există o copie a algoritmului care se termină cu succes.

Complexitatea unui algoritm nedeterminist este suma complexităților operațiilor din calea cea mai scurtă (adică de complexitate minimă) care termină algoritmul cu succes -> **complexitate angelică**

NP - COMPLETITUDINE

Algoritm tractabil => complexitate polinomială - pentru dimensiunea n a problemei, complexitatea algoritmului este

$O(n^k)$, cu k constant)

Problemă tractabilă determinist / nedeterminist => poate fi rezolvată folosind un **algoritm determinist / nedeterminist tractabil**

Problemă intractabilă => toți algoritmii determiniști de rezolvare au complexitate suprapolinomială

$\Omega(k^n)$, k constant)

Dec = mulțimea problemelor de decizie

$P = \{ Q \in Dec \mid$ există cel puțin un algoritm determinist tractabil care rezolvă problema $Q \}$

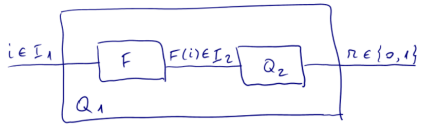
$NP = \{ Q \in Dec \mid$ există cel puțin un algoritm nedeterminist tractabil care rezolvă problema $Q \}$

$P \subseteq NP$

Problema reductibilă în timp polinomial

Def: Fie $Q_1, Q_2 \in Dec$. Spunem că problema Q_1 este reductibilă în timp polinomial la problema Q_2, relație notată $Q_1 \leq_p Q_2$, dacă există un algoritm determinist tractabil F care transformă problema Q_1 în problema Q_2.

F transformă datele $i \in I_1$ ale problemei Q_1 în date $F(i) \in I_2$ ale problemei Q_2 astfel încât: $\forall i \in I_1$ avem $Q_1(i) = 1 \Leftrightarrow Q_2(F(i)) = 1$.



$Q_1 \leq_p Q_2$

(Q_1 este reductibilă în timp polinomial la Q_2)

Proprietăți

Prop 1: Dacă $Q_2 \in P$ și $Q_1 \leq_p Q_2$ atunci $Q_1 \in P$.

Prop 2: Relația $\leq_p$ este tranzitivă:

$\forall Q_1, Q_2, Q_3 \in Dec$, dacă $Q_1 \leq_p Q_2$ și $Q_2 \leq_p Q_3$ atunci $Q_1 \leq_p Q_3$.

NP- dură

Def: O problemă $Q \in Dec$ s.m. NP- dură dacă $\forall Q' \in NP$ avem $Q' \leq_p Q$.

Formatted: Font: 10 pt

Deleted: /

NPD = mulțimea problemelor NP-dure

NP-completă

Def: O problemă $Q \in Dec$ s.n. NP-completă dacă Q este NP-dură și $Q \in NP$.

NPC = mulțimea problemelor NP-complete -> cele mai dificile probleme din clasa NP

- problemele din clasa P sunt cele mai simple probleme din clasa NP

Teoreme

(T1) $Q_2 \in NPD \iff \exists Q_1 \in NPC$ astfel încât $Q_1 \leq_p Q_2$.

(T2) $\forall Q_1, Q_2 \in NPC$ avem $Q_1 \leq_p Q_2$ și $Q_2 \leq_p Q_1$.
(Problemele din NPC sunt echivalente în raport cu relația $\leq_p$)

(T3) Dacă $NPC \cap P \neq \emptyset$ atunci $P = NP$.

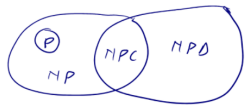
(C1) Dacă $NPC \cap P \neq \emptyset$ atunci $NPC \subseteq P$.

(T4) Dacă $NPD \cap P \neq \emptyset$ atunci $P = NP$.

(C2) Dacă $NPD \cap P \neq \emptyset$ atunci $NPC \subseteq P$.

Important!!

Știm că:
- $P \subseteq NP$
- $NPC = NPD \cap NP$



Teorema Cook

Formulă propozițională în formă normală conjugată (CNF) = formula de forma:

- variabilă = simbol care poate fi asociat unei unice valori din {0, 1}
- literal = variabilă | ¬ variabilă
- termen = literal [∨ literal]*
- formulă = termen [∧ termen]*

¬ = not, ∨ = sau logic, ∧ = și logic

O formulă CNF = **satisfiabilă** => există o asociere a variabilelor formulei F la valori din {0, 1} astfel încât valoarea lui F să fie 1.

Ex: $F = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$

$x_1 = 1$, $x_2 = 0$, $x_3 = 0$ $\Rightarrow F(1,0,0) = (1 \vee 0 \vee 0) \wedge (0 \vee 1 \vee 0) = 1 \wedge 1 = 1$

F este CNF și F este satisfiabilă

SAT = "Fiind dată o formulă CNF F, să se stabilească dacă F este satisfiabilă"

n este numărul variabilelor din formulă considerăm că numărul termenilor formulei este $m = O(n^k)$.

SAT = CNF - SAT

Teorema Cook

$$SAT \in P \iff P = NP$$

Complexitatea algoritmului

Complexitatea algoritmului N-SAT este:
$O(n^k \cdot m \cdot n^c) = O(n^{k+c+1})$
- n^k: număr termeni
- m: număr variabile
- n^c: complexitatea căutării unui literal într-un termen

Formatted: Font: 10 pt

Algoritmul de mai jos este un algoritm nedeterminist tractabil care rezolvă problema SAT

```
N_SAT(F)
    Var = variabile(F)
    for-each x ∈ Var
        x = choice({0,1})
(*)
```

```
(*)
// verificare satisfacere F
for-each term ∈ F
    s_term = 0
    for-each x ∈ Var
        if((x ∈ term ∧ x = 1) ∨ (¬x ∈ term ∧ x = 0))   // condiție if
            s_term = 1
            break
    if s_term = 0
        fail
success
```

13

SAT este NP-completă

(SAT ∈ NPC)

Metode de a arăta că o problemă Q este NP-dură

Metoda 1: $\forall Q' \in NP$ avem $Q' \leq_p Q$ (definiția unei probleme NP-dure)

Metoda 2: Se arată că o problemă Q'' NP-dură sau NP-completă îndeplinește proprietatea: $Q'' \leq_p Q$

Demonstrație că o problemă este NP-completă

Pas 1: Se arată că $Q \in NP$ adică se construiește un algoritm nedeterminist tractabil care să rezolve problema Q.

Pas 2: Se arată că problema Q este NP-dură.

PROBLEMA K-CLICA

Definiție K-Clică

Def: Fie $G = (V, E)$ un graf neorientat. S.n. K-clică a grafului G un subgraf $G' = (V', E')$ complet al lui G, astfel încât $card(V') = K$ (G' este subgraf complet dacă $\forall u, v \in V'$ avem $(u, v) \in E'$).

Problema K-Clică

"Fiind date un graf neorientat $G = (V, E)$ și un număr întreg $K > 0$ să se stabilească dacă G are o K-clică".

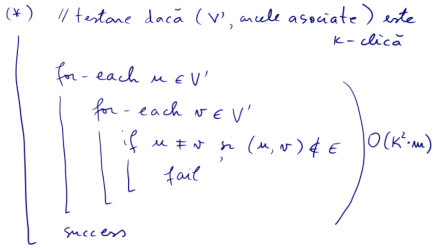
(L1) K-Clică ∈ NP

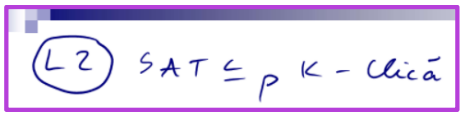
Algoritm nedeterminist ce rezolvă problema

```
N_K_Clică(G,K) // G = (V,E)        m = card(E), n = card(V)
    V' = ∅
    for i = 1..K // generare soluție
        u = choice(V)
        if u ∈ V'
            fail
(*)     V' = V' ∪ {u}                                   O(K²)
```

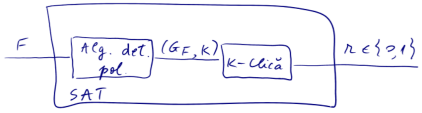
7

Formatted: Font: 10 pt





F este satisfiabilă <=> G_F are o K - clică



SAT ≤p K - Clică

K - Clică ∈ NPC.

Problema K-Acoperire

K-acoperire definiție

Def: Fie G = (V, E) un graf neorientat și V' ⊆ V. V' s.n. K - acoperire a lui G dacă:

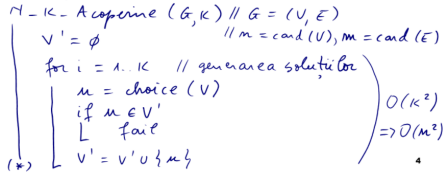
- ∀ (u,v) ∈ E avem u ∈ V' și/sau v ∈ V'
- card (V') = K.

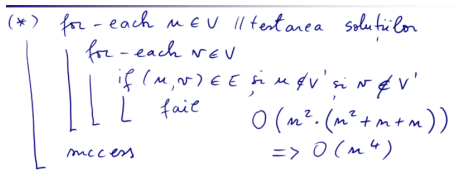
Problemă

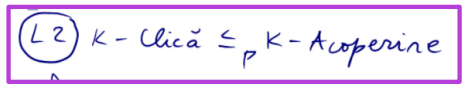
"Fiind date un graf neorientat G și un număr întreg K > 0 să se stabilească dacă G are o K - acoperire"

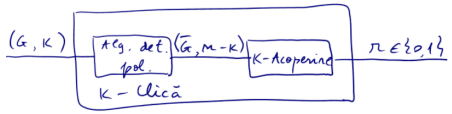
K - Acoperire ∈ NP

Algoritm nedeterminist tractabil care rezolvă problema K-Acoperire









K - Acoperire ∈ NPC.

Problema Q-Sume

Sets(n) = mulțimea tuturor mulțimilor formate cu numerele 1,2, …, n (mulțimea vidă nu este inclusă)

Problema q-Sume

Formatted: Font: 10 pt

"Fiind date o colecție $A = \{A_1, A_2, ..., A_n\}$ de n numere întregi pozitive și un număr întreg $q > 0$ să se stabilească dacă $\exists S \in Sets(n)$ astfel încât $q = \sum_{i \in S} A_i$".

⇔ *există o subcolecție a colecției A cu suma q?*

q - Sume ∈ NP

Algoritm nedeterminist tractabil care rezolvă problema q-Sume

```
N_q_Sume(A, n, q)
    S = ∅
    Suma = 0
    for k = 1..n
        i = choice({1,2,..,n})
        if i ∈ S
            fail
        S = S ∪ {i}
        Suma = Suma + A_i
        if Suma = q
            success
    fail
```

$O(n^2)$

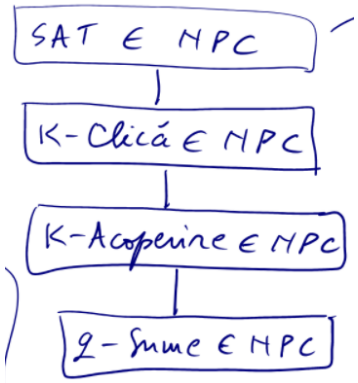
21

se de probleme

K - Acoperire ≤p q - Sume

q - Sume ∈ NPC

Arborele problemelor din NPC studiate



Problema Turnurilor din Hanoi

- se consideră n discuri de mărimi diferite
- să se mute discurile de pe turnul X pe turnul Y folosind turnul Z
- discurile sunt așezate inițial pe turnul X de jos în sus de la cel mai mare spre cel mai mic
- în timpul rezolvării un disc poate fi pus: pe un turn fără discuri, pe un turn cu discuri, dar respectând regula: **un disc se pune doar pe un disc mai mare decât el**

Algoritm:

```
Hanoi(n, X, Y, Z)
    if (n ≥ 1)
        Hanoi(n-1, X, Z, Y)
        Mută disc de pe X pe Y
        Hanoi(n-1, Z, Y, X)
```

Observație!! - Problema turnurilor din Hanoi nu poate fi rezolvată cu un algoritm determinist în timp polinomial

Ierarhie spațiu-timp a problemelor

- din punct de vedere al **timpului** de execuție o operație elementară are complexitate = **O(1)**

Formatted: Font: 10 pt

Formatted: Font: 11 pt

- **complexitate spațială** - d<=n se stochează folosind **O(lgn) biți**
- spațiul ocupat de datele de intrare nu participă la calculul complexităților la calculul complexității spațiale a algoritmului

- timp(Alg, i) este timpul consumat de calculul Alg(i);
- spatiu(Alg, i) este spațiul consumat de calculul Alg(i).

$$Alg_Q^T(f) = \{ Alg \in Alg_Q \mid \forall i \in I \text{ avem } timp(Alg, i) = O(f(n)) \}$$

$$Alg_Q^S(f) = \{ Alg \in Alg_Q \mid \forall i \in I \text{ avem } spatiu(Alg, i) = O(f(n)) \}$$

Clasele de probleme rezolvabile prin algoritmi determiniști cu limitare f temporală, respectiv spațială

$$TIME(f) = \{ Q \mid Alg_Q^T(f) \neq \emptyset \}$$

$$SPACE(f) = \{ Q \mid Alg_Q^S(f) \neq \emptyset \}$$

Cazuri particulare

$$P = PTIME = \bigcup_{p \text{ polinom}} TIME(p)$$

$$PSPACE = \bigcup_{p \text{ polinom}} SPACE(p)$$

$$LOGSPACE = \bigcup_{k \geq 1} SPACE(k \cdot \lg n)$$

unde p este un polinom cu valori pozitive

GAP = problema accesibilității unui graf

"Fiind date un graf neorientat $G = (V, E)$ și două noduri $i, j \in V$ să se stabilească dacă există drum în G între i și j".

$$GAP \in SPACE(\lg^2 n)$$

Algoritm determinist care rezolvă problema GAP într-un spațiu O(lg^2(n))

```
GAP_S(G, i, j) // G = (V, E), n = card(V)
    for h = 0 .. n-1
        if drum(i, j, h) return 1
    return 0
```

8

```
drum(i, j, lung)
    if lung = 0
        return i = j
    if lung = 1
        return (i, j) ∈ E
    l = lung / 2
    for k = 1 .. n
        if drum(i, k, l) și drum(k, j, lung - l)
            return 1
    return 0
```

9

Explicație complexitate spațială

- Lungimea maximă a lanțului de apeluri recursive este lg n. Fiecare apel consumă spațiu pentru 5 variabile (i, j, lung, k, l).
- Deoarece se lucrează cu numere ≤ n, spațiul ocupat la fiecare apel este O(lg n).
- În total funcția drum consumă $O(\lg^2 n)$ biți.

=> complexitatea spațială pentru funcția drum este $O(\lg^2 n)$ biți.

10

- datele G, i, j nu contribuie la complexitatea spațială

Clase de probleme rezolvabile nedeterminist

Formatted: Font: 10 pt

Fie N_Alg un algoritm nedeterminist cu m instrucțiuni. S.n. spațiul de stare al algoritmului pentru datele $i \in I$ un graf orientat $G(i) = (V, E, s_0, s_f)$, unde nodurile din mulțimea V reprezintă stări ale algoritmului, iar arcele din mulțimea E tranziții între stări.

Stare - corespunde valorilor curente ale variabilelor

Tranziție - depinde de instrucțiunea executată în stare n

s_0 = starea inițială

s_f = starea corespunzătoare terminării cu succes a algoritmului.

Notații

cale(N_Alg, i): drumurile simple $s_0 \dots s_f$ din graful G(i);

stări(d): stările de pe o cale $d \in$ cale(N_Alg, i);

timp_cale(d): timpul total necesar execuției tuturor prelucrărilor din calea d;

spațiu_stare(s): spațiul consumat de starea s, excluzând datele nemodificabile ale algoritmului;

$$N\text{-}timp(N\text{-}Alg, i) = \min\{timp\text{-}cale(d) \mid d \in cale(N\text{-}Alg, i)\}$$

$$N\text{-}spațiu(N\text{-}Alg, i) = \min\{spațiu\text{-}cale(d) \mid d \in cale(N\text{-}Alg, i)\}$$

$$spațiu\text{-}cale(d) = \max\{spațiu\text{-}stare(s) \mid s \in stări(d)\}$$

N_timp, N_spațiu - cum se calculează complexitatea temporală și spațială pt un algoritm nedeterminist

$$N\text{-}Alg_Q^T(f) = \{Alg \in N\text{-}Alg_Q \mid \forall i \in I \text{ avem } N\text{-}timp(Alg, i) = O(f(n))\}$$

$$N\text{-}Alg_Q^S(f) = \{Alg \in N\text{-}Alg_Q \mid \forall i \in I \text{ avem } N\text{-}spațiu(Alg, i) = O(f(n))\}.$$

Clase de probleme rezolvabile prin algoritmi nedeterminiști cu limitare f temporală, respectiv spațială

$$NTIME(f) = \{Q \mid N\text{-}Alg_Q^T(f) \neq \emptyset\}$$

$$NSPACE(f) = \{Q \mid N\text{-}Alg_Q^S(f) \neq \emptyset\}$$

Cazuri particulare

$$NP = NPTIME = \bigcup_{p \text{ polinom}} NTIME(p)$$

$$NPSPACE = \bigcup_{p \text{ polinom}} NSPACE(p)$$

$$NLOGSPACE = \bigcup_{k \geq 1} NSPACE(k \cdot \lg n)$$

$$GAP \in NLOGSPACE$$

$$GAP \in NPTIME.$$

Algoritm nedeterminist N_GAP_S cu complexitate spațială O(lgn)

```
N_GAP_S(G, i, j) // G = (V, E), n = card(V)
    v = i
    lung = 0
    while v ≠ j
        w = choice(V)
        lung = lung + 1
        if (v, w) ∉ E sau lung ≥ n
            fail
        v = w
    succes
```



```
N_GAP_S(G, i, j) // G = (V, E), n = card
    v = i
    lung = 0
    while v ≠ j
        w = choice(V)
        lung = lung + 1
        if (v, w) ∉ E sau lung ≥ n
            fail
        v = w
    succes
```


Deleted:

Formatted: Font: 10 pt

Formatted: Font: Not Bold, Not Italic

Relații între clasele de probleme

T1

$TIME(f) \subseteq NTIME(f)$

$SPACE(f) \subseteq NSPACE(f)$

C1

$LOGSPACE \subseteq NLOGSPACE$

$PTIME \subseteq NPTIME$ (sau $P \subseteq NP$)

$PSPACE \subseteq NPSPACE$

$\forall Q$ avem $Q \in NSPACE(f) \Rightarrow Q \in TIME(c^f)$, unde c este o constantă dependentă de problemă.

$NLOGSPACE \subseteq PTIME$

$NSPACE(f) \subseteq SPACE(f^2)$

$PSPACE = NPSPACE$

T1

$NPTIME \subseteq PSPACE$

$LOGSPACE \subseteq NLOGSPACE \subseteq PTIME \subseteq NPTIME \subseteq PSPACE = NPSPACE$

Algoritmi de aproximare

- rezolvarea aproximativă a problemelor dificile de optimizare (corespunzătoare unor probleme de decizie NP-dure sau NP-complete)

Algoritm de aproximare = algoritm determinist tractabil care duce la soluții neoptimale ale problemei, dar cu un grad de calitate garantat în raport cu optimul

- $cost^*(i)$ = costul sol optime
- $cost(i)$ = costul soluției generale

Factorul de aproximare al algoritmului de aproximare

$\delta(n) = \max_{i \in I, |i|=n} \frac{cost(i)}{cost^*(i)}$, dacă problema este de minimizare

$\delta(n) = \max_{i \in I, |i|=n} \frac{cost^*(i)}{cost(i)}$, dacă problema este de maximizare

Obs: $\delta(n) \geq 1, \forall n$

Dacă funcția este mărginită superior de o constantă, avem:

$\delta = \max_{i \in I} \frac{cost(i)}{cost^*(i)}$, dacă problema este de minimizare

$\delta = \max_{i \in I} \frac{cost^*(i)}{cost(i)}$, dacă problema este de maximizare

Obs: $\delta \geq 1$.

Problema λ-arc conectivitate

λ-arc conectat - prin eliminarea oricăror λ-1 arce din G se obține un graf conex

Problemă:

"Fie $G = (V, E)$ un graf neorientat și λ - arc conectat cu n noduri, unde λ este un număr întreg, $2 \leq \lambda \leq n-1$. Să se determine un subgraf de acoperire $G_\lambda = (V, E_\lambda)$ al lui G, cu număr minim de arce, astfel încât G_λ să fie λ - arc conectat."

Formatted: Font: 10 pt
Formatted: Font: Not Bold, Not Italic
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: (Default) Arial, 10 pt
Formatted: Font: Not Bold, Not Italic
Formatted: Font: Not Bold, Not Italic
Formatted: Font: Not Bold, Not Italic
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold
Formatted: Font: Not Bold

Algoritmul de aproximare Khuller-Vishkin

a) Construcția unui subgraf de acoperire 2-arc conectat

$G = (V, E)$: graful inițial, 2-arc conectat;

$G_2 = (V, E_2)$, $E_2 \subseteq E$: graful final, 2-arc conectat;

$Arb(r)$: arborele cu rădăcina r construit în cursul parcurgerii în adâncime a lui G;

$d(u)$: momentul descoperirii nodului u în cursul parcurgerii în adâncime a grafului G;

$succs(u)$: nodurile adiacente nodului u în graful G;

$\Pi(u)$: predecesorul direct al nodului u în $Arb(r)$;

$c(u)$: culoarea nodului u;

$b(u) = \min\{d(v) \mid v$ este redescoperit pornind din u în cursul explorării grafului $G\}$;

$b_2(u) = \min\{d(v) \mid v$ este redescoperit pornind din u în cursul explorării grafului $G_2\}$;

$e(u)$: arc invers (x, y) astfel încât y este predecesor al lui x în $Arb(r)$, iar u este pe calea $x, \Pi(x), \Pi(\Pi(x)), \ldots, y$ în $Arb(r)$.

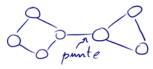
Arc de arbore = arc care face parte din arborele obținut în urma parcurgerii în adâncime a grafului

Arc invers = arc prin care se redescoperă un nod în cursul parcurgerii în adâncime a grafului

Algoritm

```
arc_2_conectat(G)
    for-each u ∈ V
        c(u) = alb
        Π(u) = null
    timp = 0
    E_2 = ∅
    fie r ∈ V ales la întâmplare
    explorare(r)
    return E_2
```

Punte = arc prin eliminarea căruia graful se deconectează.




```
explorare(u)
    d(u) = b(u) = b_2(u) = ++timp
    e(u) = (u, u) // arc invers de acoperire a punților din G_2
    c(u) = cenușiu
    for-each v ∈ succs(u)
        if c(v) = alb
            Π(v) = u
            E_2 = E_2 ∪ {(u, v)} // (u, v) arc de arbore
            explorare(v)
        (3) (2) (1)
```

```
        (3) (2) (1)
        if b(v) < b(u)
            b(u) = b(v)
            e(u) = e(v)
        if d(u) < b_2(v)
            // (u, v) punte în G_2
            E_2 = E_2 ∪ {e(v)} // punctea (u, v) este acoperită cu arcul invers e(v)
        b_2(v) = b(v)
    (3) (2)
    b_2(u) = min{b_2(u), b_2(v)}
```

```
(3) (2) else // v este redescoperit
    if Π(u) ≠ v // (u, v) arc invers
        if d(v) < b(u)
            b(u) = d(v)
            e(u) = (u, v)
c(u) = negru
```

Formatted: Font: 10 pt

Formatted: Font: Not Bold, Not Italic

Formatted: Font: Not Bold

Formatted: Font: Not Bold

Formatted: Font: Not Bold, Not Italic

Complexitate temporală = complexitate DFS = O(n + m), unde n = card(V), m = card(E)

b) Construcția unui subgraf de acoperire λ-arc conectat

- se pleacă de la un subgraf de acoperire $G_2 = (V, E_2)$ 2-arc conectat al lui G;
- se adaugă iterativ noi arce din $\bar{E}$ la arcele din E_2, la fiecare pas mărindu-se cu 1 factorul de conectivitate al subgrafului;
- în final, G_λ va fi subgraf de acoperire λ - arc conectat.

Pădure de acoperire a unui graf = mulțime de arbori care acoperă nodurile acelui graf, fiecare nod fiind acoperit o singură dată

```
arc_λ_conectat(G, λ)
    E_2 = arc_2_conectat(G)
    // construiește subgrafuri G_{i+1} = (V, E_{i+1})
    // (i+1) - arc conectate, i = 2, λ-1
    for i = 2..λ-1
        E' = E \ E_i
        A_{i+1} = arce_acoperire(V, E')
        // A_{i+1} sunt arcele pădurii de acoperire
        // a grafului (V, E')
        E_{i+1} = E_i ∪ A_{i+1}
    return E_λ
```

23

```
arce_acoperire(V, E')
    // parcurgere în adâncime a grafului (V, E')
    for-each u ∈ V
        c(u) = alb
    A = ∅
    for-each u ∈ V
        if c(u) = alb
            culege_arce(u, E')
    return A
```

24

```
culege_arce(u, E')
    c(u) = culoare
    for-each v ∈ succs(u, E')
        if c(v) = alb
            A = A ∪ {(u, v)}
            culege_arce(v, E')
```

Complexitate algoritmului

$O((\lambda - 1) \cdot (n + m))$, $n = card(V)$, $m = card(E)$.

Factorul de aproximare

$$\delta < 2 - \frac{1}{\lambda}$$

Duritatea algoritmilor de aproximare

Def: Fie Q o problemă pentru care dacă există algoritmi de aproximare cu factorul de aproximare $\delta(n) \leq f(n)$ atunci P = NP. Spunem că aproximarea problemei Q este dură sau, alternativ, că factorul de aproximare $\delta(n) = f(n)$ este dur.

2

Problema colorării unui graf

X(G) = numărul minim de culori cu care pot fi colorate nodurile grafului astfel încât pentru orice muchie (u,v) avem c(u) != c(v), unde c(x) este culoarea nodului x

Problemă

" Fiind dat un graf neorientat $G = (V, E)$ să se determine numărul cromatic al grafului G ($\chi(G)$)".

Problema K-Colorare

" Fiind date un graf neorientat $G = (V, E)$ și un număr întreg $k > 0$ să se stabilească dacă $\chi(G) = k$".

4

Formatted: Font: 10 pt

Formatted: Font: Not Bold, Not Italic

Formatted: Font: Not Bold, Not Italic

Formatted: Font: Not Bold, Not Italic

Formatted: Font: Not Bold, Not Italic

1 - Colorare $\in P$

2 - Colorare $\in P$

Problema K - Colorare $\in NPC$ pentru $K \geq 3$.

(T2) Dacă există un algoritm de aproximare cu $\delta < \frac{4}{3}$ pentru colorarea oricărui graf $\tilde{G}$ cu numărul cromatic $\chi(\tilde{G}) \geq 2$, unde $2 \geq 1$ este o constantă, atunci $P = NP$.

Def: Fie A o mulțime numărabilă și $M \subseteq \mathbb{N}$ o mulțime echipotentă cu ea. O numerotare a mulțimii A este o funcție bijectivă $a : M \to A$, $a(k) = a_k, \forall k \in M$, iar $A = \{a_k \mid k \in M\}$.

Orice mulțime numărabilă are cel puțin o numerotare.

Orice mulțime infinită și numărabilă este echipotentă cu N.

Orice mulțime finită este numărabilă.

Def: O mulțime infinită și numărabilă s.n. ∞ - numărabilă.

Def: O mulțime infinită și nenumărabilă s.n. ∞ - nenumărabilă.

Prop 1: O submulțime infinită a unei mulțimi ∞ - numărabile este ∞ - numărabilă.

Fie A o mulțime ∞ - nenumărabilă și B o mulțime ∞ - numărabilă astfel încât $B \subset A$. Atunci $A \setminus B \neq \emptyset$.

Def: $\text{Hom}(E_1, E_2) = \{f : E_1 \to E_2\}$

Prop 3:

Mulțimea $\text{Hom}(\mathbb{N}, \mathbb{N})$ este ∞ - nenumărabilă.

Mașină Turing (bine că nu dăm așa ceva că nici la restanță nu mă duceam)

Def: Notăm cu P_{ij} mulțimea programelor având ca intrare un tuplu din $\mathbb{N}^i$ și ca ieșire un tuplu din $\mathbb{N}^j$, prin program înțelegând un algoritm calculabil în sens Turing. Există o mașină de calcul (o mașină Turing) care pentru intrări valide calculează în timp finit ieșirile programului.

Mulțimea P_{ij} este ∞ - numărabilă.

O procedură efectivă = mulțime finită de instrucțiuni, finite ca dimensiune a reprezentării, a căror execuție se termină în timp finit și implică o cantitate finită de date pentru a obține un rezultat, mereu același pentru date identice.

O funcție f: I -> O s.n. efectiv calculabilă dacă există o procedură efectivă capabilă săs calculeze valoarea funcției f pentru fiecare valoare x din I.

Calculabilitate

O mulțime A este echipotentă cu o mulțime B dacă există o funcție f: A -> B bijectivă.

O mulțime se numește numărabilă dacă este echipotentă cu o submulțime a lui N.

Formatted: Font: 10 pt

Formatted: Font: Not Bold, Not Italic

Formatted: Font: Not Bold

Formatted: Font: (Default) Arial, 10 pt

Formatted: Font: Not Bold

Formatted: Font: Not Bold

Formatted: Font: Not Bold

Formatted: Font: Not Bold

Teza Church - Turing

O funcție este efectiv calculabilă <=> este Turing - calculabilă.

Teză = propoziție considerată adevărată, dar care nu a putut fi demonstrată ca fiind adevărată sau falsă.

O mulțime recursivă $A \subseteq \mathbb{N}$ are proprietatea că se poate decide în mod automat (folosind un program din $P_{1,1}$) dacă o valoare oarecare $n \in \mathbb{N}$ aparține mulțimii A.

Mulțime recursiv numărabilă

Def (*): O mulțime $A \subseteq \mathbb{N}$ s.n. recursiv - numărabilă dacă există o funcție recursivă $f: A \to \mathbb{N}$.

Def (**): O mulțime $A \subseteq \mathbb{N}$ s.n. recursiv - numărabilă dacă există un program $Q \in P_{0,1}$ care la fiecare apel să calculeze un element din A. Q s.n. generator al lui A.

O mulțime recursivă este recursiv-numărabilă.

Există mulțimi:

a) recursive;
b) care sunt recursiv - numărabile, dar nu sunt recursive;
c) care nu sunt recursive nici recursiv - numărabile.

Decidabil / semidecidabil / nedecidabil !!

Def: Fie $T: \mathbb{N} \to \{0,1\}$ un predicat și $M_T = \{m \in \mathbb{N} / T(m) = 1\}$ mulțimea de adevăr a lui T.

Decidabil

T s.n. decidabil dacă M_T este recursivă

Semidecidabil

T s.n. semidecidabil dacă M_T este recursiv - numărabilă, dar nu este recursivă;

Nedecidabil

T s.n. nedecidabil dacă M_T nu este nici recursivă și nici recursiv - numărabilă.

Exemplu problemă decidabilă

Fie $n \in \mathbb{N}$. Să se stabilească dacă n este număr prim.

Explicație: se poate construi un algoritm determinist care să răspundă în timp finit dacă n este număr prim sau nu. Prin numire: și în caz afirmativ și în caz negativ răspunsul se obține în timp finit.

21

Exemplu problemă semidecidabilă

Fie A un program și x o intrare a lui A. Calculul A(x) se termină? (Problema terminării programelor).

Explicație

```
B(A,x)
  if A(x) se termină
    return 1
  return 0
```

- B(A,x) întoarce 1 dacă A(x) se termină;
- B(A,x) nu se termină dacă A(x) nu se termină.
- Se obține rezultatul în timp finit doar pe caz afirmativ.

22

Problemă nedecidabilă

Fie A un program și x o intrare a lui A. Calculul A(x) nu se termină? (Problema "neterminării" programelor).

Explicație

```
B(A, x)
  if A(x) nu se termină
    return 1
  return 0
```

- Nu se obține rezultat în timp finit pe caz afirmativ.

23

Problemele decidabile pot fi rezolvate automat (folosind calculatoarele din ziua de astăzi)

Problemele semidecidabile și nedecidabile nu pot fi rezolvate automat.

Dacă $A \subseteq \mathbb{N}$ este recursiv - numărabilă și $B = \mathbb{N} \setminus A$ este recursiv - numărabilă atunci A este recursivă.

Teorema Rice

Def: O proprietate a programelor din $P_{1,1}$ este o mulțime de programe cu o anumită caracteristică comună.

Ex:

$$Pow2 = \{P \in P_{1,1} \mid P(n) = n^2, \forall n \in \mathbb{N}\}$$

Def: Spunem că 2 programe $P, Q \in P_{1,1}$ sunt echivalente computațional și scriem $P = Q$ dacă pentru orice intrare $n \in \mathbb{N}$ amândouă programe dau același rezultat sau amândouă nu se termină.

Def: O proprietate $Prop$ a programelor din $P_{1,1}$ s.n. extensională, nebanală dacă:

a) nebanală: $Prop \neq \emptyset$

b) extensională: $\forall P, Q \in P_{1,1}$ astfel încât $P = Q$ avem $P \in Prop \iff Q \in Prop$.

Obs:

$Pow2$ este extensională, nebanală.

14

Def: O proprietate $Prop$ a programelor din $P_{1,1}$ este decidabilă dacă

$\exists Test_{Prop} \in P_{1,1}$ astfel încât:

$$Test_{Prop}(m) = \begin{cases} 1, & P_m \in Prop \\ 0, & P_m \notin Prop \end{cases}$$

(T) (Rice)

O proprietate extensională, nebanală a programelor din $P_{1,1}$ nu este decidabilă.

Formatted: Font: 10 pt

Page 2: [1] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 2: [2] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 2: [3] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 2: [4] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 2: [5] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 2: [6] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 2: [7] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 2: [8] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 1: [9] Formatted	Stefania Vilcea	31/01/2023 11:40:00

Font: 10 pt

Page 3: [10] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [11] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [12] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [12] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [12] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [13] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [13] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [14] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [14] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [14] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [15] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [16] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [17] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [18] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [19] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [20] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [21] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [22] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: (Default) Arial, 10 pt, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [23] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [24] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [25] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [26] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [27] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [28] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [29] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [30] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [31] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [32] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: 10 pt, Not Bold

Page 3: [33] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [34] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [35] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [36] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [37] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: 10 pt, Not Bold

Page 3: [38] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [38] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 3: [39] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 5: [40] Deleted	**Mihnea-Stefan Sandulache**	**09/11/2023 15:50:00**

Page 5: [41] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 1: [42] Formatted	**Stefania Vilcea**	**31/01/2023 11:40:00**

Font: 10 pt

Page 6: [43] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [44] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [45] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: 10 pt, Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [46] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [47] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [48] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [49] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [50] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [51] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [52] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [53] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [54] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [55] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [56] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [57] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [57] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [57] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [58] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [59] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [60] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [61] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [62] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [62] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [63] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [63] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [64] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [65] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [66] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [67] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [68] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [69] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [70] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 6: [71] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [72] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [72] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [73] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [74] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [75] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [76] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: (Default) Arial, 10 pt, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [77] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [78] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [79] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [80] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [81] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [82] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [83] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [84] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [85] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [85] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [85] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [86] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [86] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [86] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [87] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [87] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [88] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [89] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 7: [90] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 1: [91] Formatted	**Stefania Vilcea**	**31/01/2023 11:40:00**

Font: 10 pt

Page 8: [92] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [93] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [94] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [95] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [96] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [96] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [96] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [97] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [97] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [97] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [98] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [98] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [99] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [100] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [101] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [102] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [102] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [102] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [102] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [103] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [103] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [103] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [103] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [104] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [105] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [106] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [107] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [108] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [109] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [110] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [111] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [112] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [113] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [114] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [115] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [116] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [117] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [118] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [119] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [120] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [121] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [122] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [123] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [124] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [125] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [126] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [127] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Italic, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [128] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [129] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [130] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [131] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 8: [132] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 1: [133] Formatted	**Stefania Vilcea**	**31/01/2023 11:40:00**

Font: 10 pt

Page 9: [134] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [135] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [136] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [137] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [138] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [139] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [140] Formatted	**Tarau Alex**	**29/01/2024 17:48:00**

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [141] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [142] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [143] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [144] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [145] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [146] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [147] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [148] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [149] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [150] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [151] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [152] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [153] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [154] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [155] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [156] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [157] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [158] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [159] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [160] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [161] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [162] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [163] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [164] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [165] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [166] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 9: [167] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 1: [168] Formatted	Stefania Vilcea	31/01/2023 11:40:00

Font: 10 pt

Page 10: [169] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [170] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [171] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [172] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [173] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [174] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [175] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [176] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [177] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [178] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [179] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [180] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [181] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [182] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [183] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [184] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [185] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [186] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [187] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [188] Formatted	Tarau Alex	29/01/2024 17:48:00

Font: Not Bold, Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [189] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [190] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [191] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [192] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [193] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [194] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [195] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [196] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))

Page 10: [197] Formatted	Tarau Alex	29/01/2024 17:48:00

Pattern: Clear (Custom Colour (RGB(255,200,156)))